

## [부록] 붙임1 SBOM 작성 가이드

결과보고서 붙임1 소프트웨어 자재명세서 작성 안내

### I. 왜 쓰나요

**SBOM(Software Bill of Materials, 소프트웨어 자재명세서)**은 내 프로그램이 어떤 오픈소스로 만들어졌는지 적은 성분표입니다. 식품 포장 뒷면의 원재료 표시를 떠올리시면 이해가 빠릅니다. 어떤 재료가 얼마나 들어갔는지 밝혀 두면, 나중에 문제가 생겼을 때 어디를 살펴야 하는지 바로 알 수 있습니다.

처음 작성해 보시는 분이 많을 텐데, 완벽하게 쓰지 못했다고 해서 그 정확도로 점수를 매기지는 않습니다. 라이선스 충돌 여부는 2차 평가 때 전문기관이 소스코드를 직접 분석해 별도로 검증하므로, 이 표에서 모든 것을 잡아내실 필요는 없습니다. 다만 성실하게 작성하신 정도는 오픈소스SW 이해도를 살펴보는 참고자료가 됩니다.

최근 소프트웨어 공급망 보안이 중요해지면서 공공기관 납품이나 기업 간 거래에서 SBOM 제출을 요구하는 경우가 빠르게 늘고 있습니다. 개발자로 일하시면 언젠가는 마주치게 될 문서인 만큼, 대회를 통해 한 번 직접 만들어 보시는 경험 자체가 이 항목의 목적입니다. 부담 없이 작성해 보시기 바랍니다.

### II. 무엇을, 어떤 순서로 적나요

내 프로젝트가 남이 만든 어떤 오픈소스를 가져다 썼는지 적습니다. 원재료 표시에 “우리 가게에서 만든 반죽”을 적지 않듯이, 이번 출품작을 위해 팀이 직접 짠 소스코드는 적지 않습니다. 출품작 자체의 정보(저장소 주소, 라이선스)는 결과보고서 본문에 이미 기재하시기 때문에 여기서 반복하실 필요가 없습니다.

한 가지 예외가 있습니다. 팀이 직접 만들었더라도 npm, PyPI 등에 별도 패키지로 공개 배포한 모듈이라면, 그 모듈은 이름·버전·라이선스·저장소가 독립적으로 존재하는 어엿한 오픈소스이므로 표에 적으셔도 됩니다. 요컨대 기준은 “누가 만들었나”가 아니라 “출품작 바깥에 독립된 패키지로 존재하는가”입니다.

개수는 10개 안팎이면 충분하며, 우선순위가 높은 것부터 위에서 아래로 채우시면 됩니다.

#### [그림] 붙임1 SBOM — 무엇을, 어떤 순서로 적나요

아래 순서대로 위에서부터 채우시면 됩니다. 10개 안팎이면 충분합니다.

1

##### 핵심 기능을 담당하는 라이브러리

그것이 빠지면 프로그램이 돌아가지 않는 것

2

##### 주요 프레임워크 · SDK

React, Spring, PyTorch, AWS SDK 등

3

##### 핵심 빌드 · 실행 도구

Vite, Gradle, Docker 등

**GPL · AGPL · LGPL**이 붙은 라이브러리는 순서와 상관없이 가장 먼저 적어 주세요.  
다른 조건보다 특별히 신경 써야 하는 라이선스입니다.

##### 적는 것

남이 만들어 공개한 오픈소스  
내 프로젝트가 가져다 쓴 것  
실제로 설치·포함된 것

##### 적지 않는 것

우리 팀이 이번에 짠 소스코드  
AI 모델 → 붙임2에 기재  
쓰다가 뺀 라이브러리  
단, 팀이 만들어 npm·PyPI 등에 별도 패키지로 공개 배포한 모듈은 적어도 됩니다

전부 적지 않아도 됩니다  
나머지는 저장소로 확인합니다

후보가 10개를 넘을 때는 아래 순서로 채웁니다. 본 서식은 요약본이므로 모든 것을 담을 필요는 없으며, ①부터 순서대로 채우다가 10개가 차면 멈추시면 됩니다.

| 순서 | 선별 기준                               | 개수       |
|----|-------------------------------------|----------|
| ①  | GPL · AGPL · LGPL 계열 라이선스가 붙은 라이브러리 | 가장 먼저 기재 |
| ②  | 핵심 기능 담당 — 빠지면 프로그램이 안 되는 것         | 2~4개     |
| ③  | 주요 프레임워크 · SDK                      | 2~4개     |
| ④  | 핵심 빌드 · 실행 도구                       | 남는 칸만큼   |

전체 목록을 따로 제출하실 필요는 없습니다. 심사 과정에서 확인이 필요한 경우에는 저장소에 이미 들어 있는 의존성 관리 파일(package.json, requirements.txt, build.gradle 등 개발 환경에 따라 이름이 다릅니다)을 참고합니다. 다만 이 파일들에는 각 라이브러리의 라이선스가 표시되지 않으므로, 붙임1에는 라이선스를 직접 확인해 적어 주시기 바랍니다.

### Ⅲ. 여러 개를 묶어서 적어도 되나요

같은 곳에서 나온 SDK 묶음처럼 라이선스와 공식 저장소가 동일하다면 한 줄로 묶으셔도 됩니다. 예를 들어 @aws-sdk/client-s3, @aws-sdk/client-dynamodb 등 여러 패키지를 쓰셨다면 "@aws-sdk/client-\* 외 4개"처럼 대표 이름으로 적으시면 됩니다. 다만 라이선스가 서로 다른 패키지끼리는 묶지 마시고 나누어 적어 주십시오. 라이선스 칸에 적을 값이 하나로 정해지지 않으면 검증 단계에서 확인이 어렵습니다.

### Ⅳ. 내가 쓴 목록, 어떻게 확인하나요

적어야 할 내용과 기준(10개 안팎, 우선순위)은 모든 팀이 동일합니다. 다만 자신이 쓴 오픈소스 목록을 확인하는 방법은 개발 방식에 따라 두 갈래로 나뉩니다. **npm, pip, Gradle 같은 도구로 라이브러리를 설치하며 개발했다면 방법 A, 그런 도구 없이 파일을 직접 내려받아 넣는 방식으로 개발했다면 방법 B를 따라 주십시오.**

#### 방법 A. 도구로 개발한 팀 — GitHub 화면으로 확인 (권장)

도구로 개발했다면 의존성 파일(package.json, pyproject.toml 등)이 개발 과정에서 이미 자동으로 만들어져 있습니다. 소스코드를 GitHub에 올리기만 하면 별도 작업 없이 아래 절차로 확인할 수 있습니다.

| 단계 | 할 일                                                                                                                          |
|----|------------------------------------------------------------------------------------------------------------------------------|
| 1  | 최종 코드를 GitHub에 올린(push) 뒤, 저장소 첫 화면에서 의존성 파일(package.json, pyproject.toml 등)을 클릭해 엽니다. 내가 직접 가져다 쓴 라이브러리가 버전과 함께 한 화면에 나옵니다. |
| 2  | 그 목록에서 표장의 우선순위 규칙대로 10개 안팎을 선별합니다.                                                                                          |
| 3  | 저장소의 Insights 탭 → Dependency graph를 열고, 검색창에 선별한 라이브러리 이름을 하나씩 검색해 라이선스를 확인합니다. 선별한 것만 확인하면 되므로 몇 분이면 끝납니다.                  |

Dependency graph에는 간접 의존성(Transitive)까지 수백 개가 보일 수 있으나, 불임1의 대상은 직접 가져다 쓴 것(Direct)뿐입니다. 목록 전체를 뒤질 필요 없이 1단계의 파일을 기준으로 삼으십시오. 또한 actions/checkout 등 "GitHub Actions"로 표시되는 항목은 저장소 자동화 도구로, 불임1에 적지 않습니다. 확인은 반드시 최종 push를 마친 뒤에 하십시오 — 심사가 보는 것도 바로 그 화면입니다.

터미널이 익숙하다면 아래 명령어로도 같은 목록을 볼 수 있습니다. **자기 프로젝트 폴더 안에서, 명령어 한 줄만 입력해야 합니다.**

| 개발 환경         | 명령어                                                                  |
|---------------|----------------------------------------------------------------------|
| Node.js       | <code>npm ls --depth=0</code>                                        |
| Python        | <code>pip list --format=freeze</code>                                |
| Java (Gradle) | <code>./gradlew dependencies --configuration runtimeClasspath</code> |
| Java (Maven)  | <code>mvn dependency:list</code>                                     |

주의 — ① 명령어는 자기 프로젝트 폴더 안에서 실행해야 하며, 다른 위치에서는 빈 결과나 오류가 나옵니다. ② 위 명령어 한 줄만 입력하십시오. 안내 자료의 예시 출력까지 함께 붙여 넣으면 오류가 납니다. ③ 출력에 프로젝트와 무관한 패키지가 섞여 나올 수 있으니(특히 Python), 실제 사용하는 것만 골라 적으십시오. ④ 이 출력에는 라이선스가 나오지 않으므로 라이선스는 별도로 확인해야 합니다.

## 방법 B. 도구 없이 개발한 팀 — 직접 찾아 작성

C/C++처럼 라이브러리 파일을 직접 내려받아 프로젝트에 넣는 방식으로 개발했다면 자동 목록이 없으므로, 아래 세 군데를 살펴 가져온 것을 찾아냅니다.

| 찾는 곳                                   | 나오는 것                                                                                 |
|----------------------------------------|---------------------------------------------------------------------------------------|
| 프로젝트 안의 lib/, vendor/, third_party/ 폴더 | 직접 내려받아 넣은 jar, dll, so, 헤더 파일                                                        |
| 소스코드 상단의 #include, import 문            | 표준 라이브러리가 아닌 것 = 외부에서 가져온 것                                                           |
| IDE 설정 화면                              | Visual Studio: 프로젝트 속성 → 링커 → 추가 종속성 / Eclipse: Build Path / Unity: Package Manager 창 |

찾아낸 각 항목의 버전은 내려받은 파일명(예: opencv-4.9.0.zip), 헤더·소스 상단 주석, 파일 우클릭 → 속성에서 확인하고, 셋 다 없으면 "4.x 추정"처럼 솔직하게 적습니다. 라이선스는 그 라이브러리 공식 저장소의 LICENSE 파일이 정답입니다. 사용 목적 칸에는 "DLL 동적 링크", "정적 링크", "소스 일부 복사", "script 태그로 로드"처럼 결합 방식을 함께 적어 주십시오.

특히 남의 소스코드를 복사해 프로젝트에 붙여 넣은 경우가 가장 누락되기 쉽습니다. 어느 파일에도 기록이 남지 않지만 2차 평가의 소스코드 분석에는 그대로 확인되므로, 반드시 기억해 불임1에 적어 주십시오. 아울러 불임1에 적은 내용을 THIRD\_PARTY.md 파일로 저장소 최상위에 함께 올려 두시면, 심사 과정에서 별도 문의 없이 확인이 가능해집니다.

## V. 적을 때 주의할 점

| 항목      | 이렇게 적어 주세요                                                            |
|---------|-----------------------------------------------------------------------|
| 버전      | ^18.2.0 같은 범위 표기가 아니라, 실제로 설치된 18.2.0으로 적습니다.                         |
| 라이선스    | "아파치", "ASL2" 같은 임의 표기 대신 MIT, Apache-2.0, GPL-3.0-only처럼 공식 표기를 씁니다. |
| 저장소 URL | 배포 페이지가 아니라 소스코드가 있는 공식 저장소 주소를 적습니다.                                 |
| 사용 목적   | 기능 설명과 함께 "라이브러리로 불러 씀", "소스 일부를 복사해 넣음"처럼 어떻게 붙였는지를 한마디 덧붙입니다.       |
| AI 모델   | 여기가 아니라 붙임2 「AI 모델 활용 및 라이선스 기술 명세서」에 적습니다.                           |

## VI. 작성 예시

아래는 웹 서비스 형태의 프로젝트를 가정해 채워 본 예시입니다. 세 번째 줄은 같은 벤더의 SDK를 묶어 적은 경우입니다.

| 번호 | 라이브러리명                 | 버전      | 라이선스       | 공식 저장소 URL                         | 사용 목적 및 주요 기능                  |
|----|------------------------|---------|------------|------------------------------------|--------------------------------|
| 1  | React                  | 18.2.0  | MIT        | github.com/facebook/react          | 프론트엔드 화면 렌더링 / 라이브러리로 불러 씀     |
| 2  | FastAPI                | 0.110.0 | MIT        | github.com/tiangolo/fastapi        | 백엔드 API 서버 / 라이브러리로 불러 씀       |
| 3  | @aws-sdk/client-* 외 4개 | 3.540.0 | Apache-2.0 | github.com/aws/aws-sdk-js-v3       | S3·DynamoDB 연동 / 라이브러리로 불러 씀   |
| 4  | Tesseract OCR          | 5.3.4   | Apache-2.0 | github.com/tesseract-ocr/tesseract | 이미지 속 문자 인식 / 실행 파일 호출 방식으로 사용 |

행이 부족하면 자유롭게 추가하셔도 됩니다. 반대로 사용한 오픈소스가 10개보다 적다면 있는 만큼만 적으시면 됩니다.

**문의** 오픈소스 개발자대회 운영사무국(한국오픈소스협회) [contest@oss.kr](mailto:contest@oss.kr)

근거 : 2026년 오픈소스 개발자대회 결과보고서 서식 붙임1 / 「운영규정」 제8조(소스코드의 오픈소스SW 라이선스 적용 의무)

© 한국오픈소스협회(KOSSA), 2026. 본 문서는 크리에이티브 커먼즈 저작자표시 4.0 국제 라이선스(CC BY 4.0)에 따라 이용하실 수 있습니다. 출처(한국오픈소스협회, 2026 오픈소스 개발자대회 운영사무국)를 표시하시면 복제·수정·배포 및 2차적 저작물 작성이 가능합니다. <https://creativecommons.org/licenses/by/4.0/deed.ko>